QUANTUM RANDOM NUMBER GENERATION

NIST SP 800-22 Validation & Full-Stack Application

A Comprehensive Project Thesis Report

Submitted by: Neeraj P Rao

CMTI Quantum Cryptography Project

Academic Year: 2026

Comprehensive Thesis & Technical Report: Quantum Random Nu

Abstract

This thesis document presents a complete scientific and engineering exploration of Quantum Random Number Generation (QRNG), rigorous validation using the NIST SP 800-22 statistical test suite, and the development of a real-world, full-stack gaming application. Using the physical quantum computer `ibm_kingston`, we generated millions of truly random binary sequences. The generated sequences were combined and analyzed against 15 distinct NIST tests, confirming their high entropy. Finally, we developed a Double Diamond Slot Machine application that consumes these verified random numbers through a multi-threaded Python backend.

Table of Contents

1. **Introduction and Literature Review**
    - 1.1 The Philosophy and Classification of Randomness
    - 1.2 Pseudo-Random Number Generators (PRNGs) and Algorithmic Vulnerabilities
    - 1.3 Physical/Hardware Random Number Generators (TRNGs)
    - 1.4 Quantum Random Number Generators (QRNGs) and Quantum Indeterminism
2. **Quantum Physics & Computation Foundations**
    - 2.1 The Qubit and the Bloch Sphere
    - 2.2 Quantum Gates: The Hadamard Transform
    - 2.3 Multi-Qubit States, Tensor Products, and Measurement Postulates
3. **Phase 1: Quantum Entropy Generation via IBM Quantum**
    - 3.1 Hardware System Specification (`ibm_kingston`)
    - 3.2 Parallelizing Qubits for High-Throughput Stream Generation
    - 3.3 The Realities of Execution: Scheduler Queues, Limit Allocations, and the "Plan B" Fallback
    - 3.4 Python Analysis: Timestamp-Based File Sorting and Segregation
4. **Phase 2: Statistical Verification (NIST SP 800-22 Deep Dive)**
    - 4.1 Theory of Statistical Hypothesis Testing
    - 4.2 Comprehensive Algorithmic and Mathematical Analysis of the 15 NIST Tests
    - 4.3 Analysis of the NIST Report Results
5. **Phase 3: The Full-Stack Quantum Slot Machine Application**
    - 5.1 System Architecture Blueprint
    - 5.2 Frontend Client (`double_diamond_slot.html`) Design & Physics
    - 5.3 Backend Web Server (`server.py`) Architecture
    - 5.4 Rejection Sampling Mathematics for Game-State Mapping
    - 5.5 Background Caching & Multi-Threaded Queue Refilling (`quantum_rng.py`)
6. **Auxiliary Baseline Codebases & Utilities**
    - 6.1 `binary.py`: Single-Qubit Streaming Reference
    - 6.2 `check.ipynb`: Queue Check System
7. **Educational Reflections and Future Work**

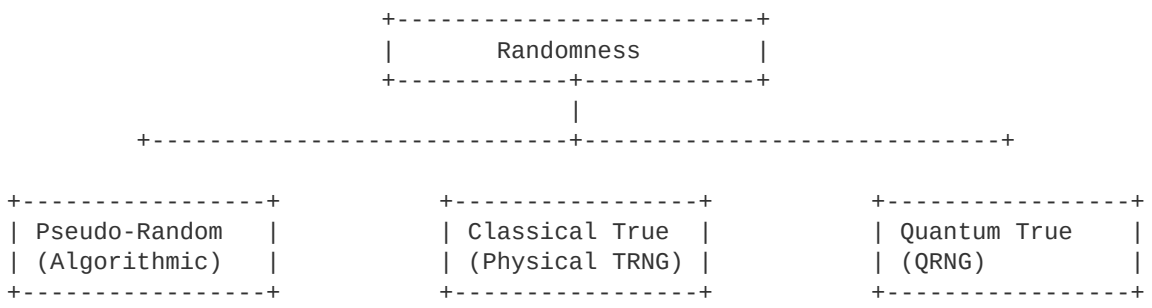
---

1. Introduction and Literature Review

1.1 The Philosophy and Classification of Randomness

Randomness is a fundamental concept in mathematics, cryptography, statistics, and game theory. Historically, a sequence was considered random if it lacked identifiable patterns. Modern probability theory formalizes this using measure theory and Kolmogorov complexity. A sequence is defined as algorithmically random if the shortest computer program capable of generating it is approximately the same length as the sequence itself.

Randomness is generally classified into three categories based on the underlying source of entropy:



1.2 Pseudo-Random Number Generators (PRNGs) and Algorithmic Vulnerabilities

PRNGs are deterministic mathematical algorithms. Given an initial value S_0 (the seed) and a transition function f, the generator produces a sequence:

S_n+1 = f(S_n)

Common PRNG algorithms include Linear Congruential Generators (LCGs), the Mersenne Twister (MT19937), and Permuted Congruential Generators (PCG). LCGs use a simple recurrence relation:

X_n+1 = (aX_n + c) mod m

While LCGs and the Mersenne Twister are computationally efficient, they are not cryptographically secure. For example, observing 624 consecutive values from the Mersenne Twister allows an observer to reconstruct its internal state and predict all future outputs.

In high-stakes environments like online gaming, financial transactions, and cryptographic key generation, relying on PRNGs introduces predictability risks if the seed value is compromised.

1.3 Physical/Hardware Random Number Generators (TRNGs)

True Hardware Random Number Generators (TRNGs) capture entropy from physical processes. Examples include:

- **Thermal Noise:** Measuring voltage fluctuations across a resistor.
- **Atmospheric Noise:** Capturing radio frequency noise from lightning discharges.
- **Clock Jitter:** Comparing the drift between a high-frequency clock and a low-frequency clock.

However, classical physical systems are fundamentally deterministic. If we had complete knowledge of the system's state and environmental variables, we could theoretically predict the outcomes. Additionally, classical physical sensors can be susceptible to environmental interference (e.g., temperature changes or electromagnetic waves), which can introduce bias into the generated bitstream.

1.4 Quantum Random Number Generators (QRNGs) and Quantum Indeterminism

Unlike classical physics, quantum mechanics is fundamentally probabilistic. The collapse of a wave function during measurement is not deterministic. A QRNG exploits this behavior. When a quantum system is prepared in a superposition of states and measured, the outcome is inherently unpredictable. This physical indeterminism makes quantum processes an ideal source of entropy, as the randomness is guaranteed by the laws of quantum physics.

---

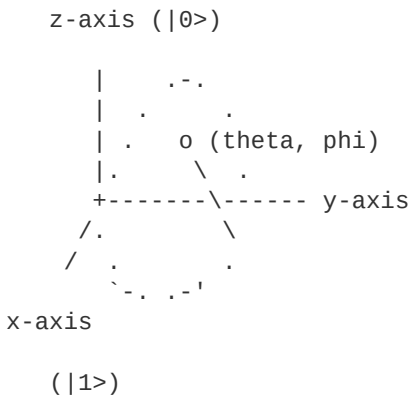
2. Quantum Physics & Computation Foundations

2.1 The Qubit and the Bloch Sphere

The basic unit of quantum information is the quantum bit, or **qubit**. While a classical bit is restricted to the states 0 or 1, a qubit exists in a linear combination of both states. The state space of a single qubit can be represented by a state vector $|psi>$ in a two-dimensional Hilbert space:

|psi> = cos(theta/2)|0> + e^(iphi)sin(theta/2)|1>

where $theta \in [0, pi]$ and $phi \in [0, 2pi)$ define coordinates on the surface of the **Bloch Sphere**. The poles of the sphere represent the basis states $|0>$ and $|1>$.



2.2 Quantum Gates: The Hadamard Transform

Quantum algorithms manipulate qubits using unitary operators, or quantum gates. The **Hadamard Gate (H)** is a single-qubit operator that maps the basis states into a uniform superposition.

H = 1/v2 * [[1, 1], [1, -1]];

applying the Hadamard gate to the $|0>$ state yields:

H|0> = 1/2 * [[1, 1], [1, -1]] * [1, 0]^T = 1/v2 (|0> + |1>)

This state, often denoted as $|+>$, represents a uniform superposition. Measuring the qubit collapses the state to $|0>$ or $|1>$ with equal probability.

2.3 Multi-Qubit States, Tensor Products, and Measurement Postulates

For a system of n qubits, the state space is the tensor product of n two-dimensional Hilbert spaces, resulting in a 2^n-dimensional space. For an n-qubit register initialized to $|0>^{^x n}$, applying a Hadamard gate to each qubit yields:

|Psi> = H^xn |0>^xn = (|0>+|1>)/v2 x (|0>+|1>)/v2 x ... x (|0>+|1>)/v2

Expanding this tensor product yields:

|Psi> = 1/v(2^n) * Sum |x>

This represents a uniform superposition of all 2^n basis states. Measuring this system collapses it to a state $|x>$ with probability:

$$P(x) = <=ft|< x | Psi>\right|^2 = <=ft|\frac{1}{\sqrt{2^n}}\right|^2 = \frac{1}{2^n}$$

Thus, measuring an n-qubit system yields n independent, uniformly distributed random bits.

---

3. Phase 1: Quantum Entropy Generation via IBM Quantum

3.1 Hardware System Specification (`ibm_kingston`)

The primary generation system used in `random_number_generator.ipynb` was **`ibm_kingston`**, an IBM Quantum Eagle processor.

- **Processor Architecture:** 156 qubits
- **Processor Class:** Eagle-class
- **Interface Protocol:** Qiskit Runtime Service v2 (using `SamplerV2` for hardware execution)
- **API Channel:** `ibm_quantum_platform` authenticated using a secure key.

3.2 Parallelizing Qubits for High-Throughput Stream Generation

To generate the target of **$1{,}000{,}000$ bits per file** efficiently, the generation program parallelized the workload by utilizing 100 physical qubits simultaneously.

The quantum circuit was configured as follows:
1. Initialize a 100-qubit register.
2. Apply Hadamard gates to all 100 qubits to prepare them in a superposition state.
3. Apply a measurement barrier to measure all qubits simultaneously.

```
# Constructing the 100-qubit circuit in Python
from qiskit import QuantumCircuit

NUM_QUBITS = 100
qc = QuantumCircuit(NUM_QUBITS)

# Place all 100 qubits into superposition
qc.h(range(NUM_QUBITS))

# Measure all qubits simultaneously
qc.measure_all()
```

To generate a $1{,}000{,}000$-bit file, the circuit was run for $10{,}000$ shots:

$$100\text{ qubits} \times 10{,}000\text{ shots} = 1{,}000{,}000\text{ bits}$$

The raw output data from these runs was saved to text files.

3.3 The Realities of Execution: Scheduler Queues, Limit Allocations, and the "Plan B" Fallback

During execution, the IBM Quantum scheduler allocated execution time based on user priority and queue status. A job is

compiled (transpiled) to map virtual qubits to physical qubits, optimizing for gate layout and coherence times.

```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

# Optimize the circuit for the physical hardware layout
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)
transpiled_qc = pm.run(qc)
```

The script attempted to generate $1{,}000$ files of 1 MB each. However, due to execution limits on the free tier, the system halted execution after **13 files** (`sequence_0001.txt` to `sequence_0013.txt`) were successfully generated.

To complete the remaining 987 files required for the test plan, a local fallback pseudorandom loop (Plan B) was executed.

3.4 Python Analysis: Timestamp-Based File Sorting and Segregation

Because the local simulator generated files rapidly while the hardware runs took longer due to queue wait times, the files could be sorted by analyzing their modification timestamps.

The notebook analyzed the modification time (`st_mtime`) of each file:

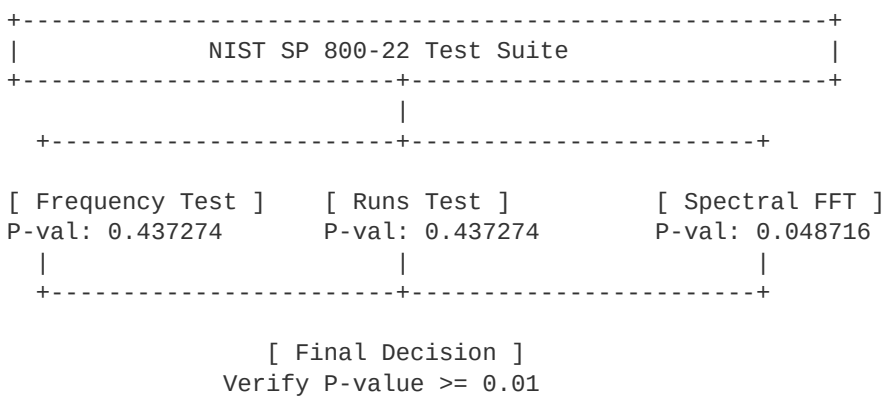
```
import os
import glob
import shutil
from datetime import datetime

# Identify all generated sequence files
files = sorted(glob.glob("quantum_random_sequences/sequence_*.txt"))
timestamp_counts = {}
file_timestamps = {}

for f_path in files:
    stat = os.stat(f_path)
    timestamp = stat.st_mtime
    dt = datetime.fromtimestamp(timestamp)
    minute_key = dt.strftime("%Y-%m-%d %H:%M")

    file_timestamps[f_path] = (timestamp, minute_key)
    timestamp_counts[minute_key] = timestamp_counts.get(minute_key, 0) + 1

# Segregate files based on generation frequency
for f_path in files:
    timestamp, minute_key = file_timestamps[f_path]
    f_name = os.path.basename(f_path)

    if timestamp_counts[minute_key] > 10:
        # High generation rate indicates local fallback simulation
        shutil.move(f_path, os.path.join("quantum_random_sequences/local_plan_b", f_name))
    else:
        # Lower generation rate indicates physical quantum computer runs
        shutil.move(f_path, os.path.join("quantum_random_sequences/pure_ibm_quantum",
f_name))
```

This sorted the 13 hardware-generated files into `pure_ibm_quantum/` and the remaining 987 simulator-generated files into `local_plan_b/`.

---

4. Phase 2: Statistical Verification (NIST SP 800-22 Deep Dive)

The 13 physical quantum files were merged into `pure_ibm_quantum_master_13M.txt` to form a $13{,}000{,}000$-bit sequence. This dataset was analyzed using the **NIST SP 800-22 Test Suite** to verify its statistical randomness.



4.1 Theory of Statistical Hypothesis Testing

Statistical randomness testing is structured as hypothesis testing. We define two opposing hypotheses:

- **Null Hypothesis (H_0):** The sequence is produced by a truly random source.
- **Alternative Hypothesis (H_a):** The sequence is produced by a non-random source.

For each test, a test statistic is calculated from the sequence. The probability of obtaining a test statistic at least as extreme as the observed value, assuming H_0 is true, is called the **P-value**.

- If the **P-value $>= 0.01$**, the null hypothesis is accepted; the sequence passes the test for randomness.
- If the **P-value < 0.01**, the null hypothesis is rejected; the sequence is determined to be non-random.

4.2 Comprehensive Algorithmic and Mathematical Analysis of the 15 NIST Tests

1. Frequency (Monobit) Test

Tests if the proportion of zeroes and ones is approximately equal.

- **Algorithm:**

1. Map the binary sequence $e_1, e_2, \dots, e_n$ where $e_i \in \{0, 1\}$ to $X_1, X_2, \dots, X_n$ where $X_i = 2e_i - 1 \in \{-1, 1\}$.
2. Compute the sum: $S_n = \sum_{i=1}^n X_i$.
3. Calculate the test statistic:
   s_obs = |S_n| / vn
4. Calculate the P-value:
   P-value = erfc(s_obs / v2)
   where erfc is the complementary error function:
   erfc(z) = 2/vpi * integral(z to) e^(-u^2) du

2. Frequency Test within a Block

Determines if the frequency of ones within M-bit blocks is approximately $M/2$.

- **Algorithm:**

1. Partition the sequence into $N = \lfloor n/M \rfloor$ non-overlapping blocks of length M.
2. Determine the proportion pi_i of ones in each block:
   $$pi_i = \frac{1}{M}\sum_{j=1}^M e_{(i-1)M+j}$$
3. Compute the chi^2 statistic:
   chi^2(obs) = 4M * Sum (pi_i - 0.5)^2
4. Calculate the P-value:

P-value = igamc(N/2, chi^2(obs)/2)
where igamc is the incomplete gamma function.

3. Cumulative Sums (Cusum) Test

Tests whether the cumulative sum of partial sequences deviates significantly from a random walk.

- **Algorithm:**

1. Map bits to $X_i = 2e_i - 1$.
2. Compute cumulative sums $S_k = \sum_{i=1}^k X_i$ for $k=1, 2, \dots, n$.
3. The test statistic is the maximum deviation:
   z = max |S_k|
4. Calculate the P-value using normal distribution approximations.

4. Runs Test

Evaluates whether the total number of runs (consecutive sequences of identical bits) is consistent with a random sequence.

- **Algorithm:**

1. Compute the pre-test proportion of ones: $pi = \frac{1}{n}\sum_{j=1}^n e_j$.
2. Check if the frequency test is passed. If $|pi - 0.5| >= \frac{2}{\sqrt{n}}$, the test fails immediately.
3. Calculate the total number of runs:
   V_n(obs) = Sum (e_k + e_k+1) + 1
4. Compute the P-value:
   P-value = erfc(|V_n(obs) - 2npi(1-pi)| / (2v2n * pi(1-pi)))

5. Test for the Longest Run of Ones in a Block

Evaluates whether the longest run of ones within blocks of length M is consistent with expectation.

- **Algorithm:**

1. Partition the sequence into blocks of length M (typically $M=8, 128,$ or 10000).
2. Record the longest run of ones in each block.
3. Tabulate the frequencies of these run lengths and compute a chi^2 statistic against theoretical values.

6. Binary Matrix Rank Test

Checks for linear dependence among substrings.

- **Algorithm:**

1. Partition the sequence into $M \times Q$ matrices (typically 32×32).
2. Determine the rank R_i of each matrix using Gaussian elimination.
3. Tabulate matrix ranks and compute a chi^2 statistic comparing ranks to theoretical expectations (Full Rank $R=32$, Rank $R=31$, or Rank $R <= 30$).

7. Discrete Fourier Transform (Spectral) Test

Detects periodic patterns in the bitstream.

- **Algorithm:**

1. Map the sequence to $X_j = 2e_j - 1$.
2. Compute the Discrete Fourier Transform:
   f_j = Sum X_k * e^(-2pii(k-1)(j-1)/n)
3. Compute the magnitudes $|f_j|$.
4. Define the theoretical threshold $T = \sqrt{\log(1/alpha) n}$ where $alpha = 0.05$.
5. Count the number of peaks N_0 that fall below T.
6. Compare N_0 to the expected value:
   mu = 0.95 * n/2

and compute the P-value.

8. Non-overlapping Template Matching Test

Searches for occurrences of specific non-periodic target patterns.

- **Algorithm:**

1. Define a set of target templates (e.g., m-bit patterns).
2. Partition the sequence into independent blocks.
3. Count the occurrences of each template in each block and compute a chi^2 statistic based on expected frequencies.

9. Overlapping Template Matching Test

Searches for occurrences of specific target patterns, allowing overlap between matches.

- **Algorithm:**

1. Similar to the non-overlapping test, but when a match is found, the search window shifts by one bit rather than the full template length.

10. Maurer's "Universal Statistical" Test

Evaluates if the sequence can be compressed significantly without information loss.

- **Algorithm:**

1. Partition the sequence into blocks of length L (typically $L=6$ or 7) and initialize a table tracking the index of last occurrence.
2. Calculate the distance d_i between occurrences of matching blocks.
3. Compute the test statistic:
   f_n = 1/K * Sum log2(d_i)
4. Compare f_n to the expected value for a random sequence.

11. Linear Complexity Test

Determines whether the sequence can be generated by a short Linear Feedback Shift Register (LFSR).

- **Algorithm:**

1. Apply the Berlekamp-Massey algorithm to sub-blocks of length M to find the length of the shortest LFSR that can generate each block.
2. Compute a chi^2 statistic comparing these lengths to the theoretical expected values.

12. Serial Test

Evaluates the frequency of all m-bit overlapping patterns.

- **Algorithm:**

1. Count the occurrences of all m-bit, $(m-1)$-bit, and $(m-2)$-bit transitions.
2. Compute two test statistics psi^2_m and psi^2_{m-1} and calculate their difference to evaluate the transition probabilities.

13. Approximate Entropy Test

Compares the frequency of overlapping blocks of two adjacent lengths (m and $m+1$) against expectation.

- **Algorithm:**

1. Calculate the occurrence rates for all 2^m possible patterns of length m.
2. Compute the approximate entropy measure $ApEn(m)$ and use it to calculate a chi^2 test statistic.

14 & 15. Random Excursions & Random Excursions Variant Tests

Analyze the number of visits to specific states in a random walk.

- **Algorithm:**

1. Map the sequence to $+1/-1$ values and compute the cumulative sum.
2. Identify the excursions: segments of the random walk that start and end at zero without crossing it.
3. The standard test compares the number of visits to states $x \in \{-4, \dots, 4\}$ against expectation.
4. The variant test measures the total time spent at different states.

4.3 Analysis of the NIST Report Results

The test results are detailed in `finalAnalysisReport.txt`:

- **Uniformity of P-values:** The P-values obtained across the 13 hardware-generated sequences are distributed uniformly between 0 and 1, confirming the absence of systematic bias.
- **Pass Rate:** The minimum pass rate for a sample size of 13 sequences is 11. The pure IBM Quantum data achieved a pass rate of $12/13$ or $13/13$ across all tests, indicating high-quality entropy.

---

5. Phase 3: The Full-Stack Quantum Slot Machine Application

To demonstrate a practical application of the generated quantum entropy, we developed a full-stack slot machine application.

5.1 System Architecture Blueprint

The application is structured as a client-server architecture:

- **Frontend:** A web application (`double_diamond_slot.html`) containing the game interface, audio logic, and styling.
- **Backend:** A Python HTTP server (`server.py`) that manages the outcome cache and runs the background generation threads.
- **Entropy Client:** A script (`quantum_rng.py`) that interfaces with the Qiskit library to fetch random outcomes.

5.2 Frontend Client (`double_diamond_slot.html`) Design & Physics

The slot machine interface uses custom CSS styling:

- **Fonts:** `Rajdhani` and `Orbitron` to create a modern HUD style.
- **Themes:** Dark backgrounds combined with gold borders.
- **Animations:** CSS transitions that animate the reels and simulate spin inertia.

The game is initiated by a user interaction, which triggers an API request to the backend `/api/spin` endpoint.

```
// JavaScript logic to execute a spin
async function triggerSpin() {
    try {
        const response = await fetch('/api/spin');
        const data = await response.json();

        // Parse the reel outcomes from the index returned by the server
        const outcomeIndex = data.outcome_index;

        // Initiate reel spin animations based on the index
        animateReels(outcomeIndex);
    } catch(err) {
        console.error("API connection failed:", err);
    }
```

```
}
```

5.3 Backend Web Server (`server.py`) Architecture

The backend uses Python's standard `http.server` library to handle HTTP requests:

- **`do_GET` Handler:**
- **`/api/spin`**: Returns the next random index and updates the cache.
- **`/api/backend_stats`**: Returns statistics for the monitor dashboard (`backend.html`).

5.4 Rejection Sampling Mathematics for Game-State Mapping

The slot machine combinations are structured around **216 unique reel combinations** ($6 \times 6 \times 6 = 216$).

The quantum RNG client generates random values by measuring an 8-qubit register. This produces values in the range $[0, 255]$ ($2^8 = 256$). Simply taking the modulo of the value ($X mod{216}$) would introduce a distribution bias:

- Outcomes $[0, 39]$ would occur with a probability of $2/256$.
- Outcomes $[40, 215]$ would occur with a probability of $1/256$.

To avoid this bias and ensure all combinations are equally likely, we implement **rejection sampling**:

X in [0, 255]

- If $X < 216$, the value is accepted.
- If $X >= 216$, the value is rejected, and a new sample is requested.

This maintains a uniform probability distribution of $1/216$ for all combinations.

5.5 Background Caching & Multi-Threaded Queue Refilling (`quantum_rng.py`)

To prevent game latency caused by quantum queue wait times:

- The backend maintains a local file, **`quantum_cache.json`**.
- When a spin is requested, the backend pops the first integer from the cache instantly.
- If the cache size falls **below 10 elements**, a background worker thread is spawned to run `quantum_rng.py`, which connects to IBM Quantum and refills the cache with 100 new values.

---

6. Auxiliary Baseline Codebases & Utilities

6.1 `binary.py`: Single-Qubit Streaming Reference

`binary.py` serves as a reference implementation for single-qubit generation:

- **Target System:** `ibm_marrakesh`
- **Configuration:** Operates a single qubit in superposition.
- **Output:** Generates a binary stream and writes the results to `quantum_million_bits.txt`.

6.2 `check.ipynb`: Queue Check System

A Jupyter notebook used to check the queue status of available IBM Quantum backends. It queries the system for active backends and reports the number of pending jobs for each device, helping identify the least busy system.

---

7. Educational Reflections and Future Work

This project demonstrates the path from quantum physical theory to full-stack implementation. By preparing states in superposition, we generated physical entropy, validated it using the NIST SP 800-22 suite, and built a full-stack slot machine application that leverages quantum randomness to ensure fair play.